



Reverse every K-element Sub-list (medium)

We'll cover the following

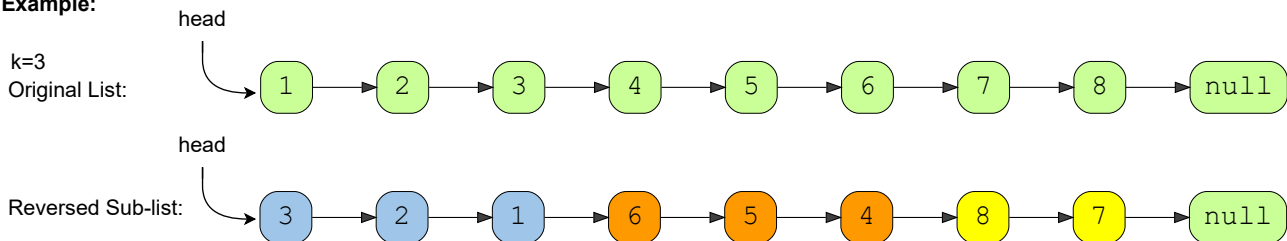
- Problem Statement
 - Try it yourself
 - Solution
 - Code
 - Time complexity
 - Space complexity

Problem Statement

Given the head of a LinkedList and a number 'k', **reverse every 'k' sized sub-list** starting from the head.

If, in the end, you are left with a sub-list with less than 'k' elements, reverse it too.

Example:



Try it yourself

Try solving this question here:

Java

Python3

JS

C++

```
1 import java.util.*;
2
3 class ListNode {
4     int value = 0;
5     ListNode next;
6
7     ListNode(int value) {
8         this.value = value;
9     }
10 }
11
12 class ReverseEveryKElements {
```



Tr



```

13
14 public static ListNode reverse(ListNode head, int k) {
15     // TODO: Write your code here
16     return head;
17 }
18
19 public static void main(String[] args) {
20     ListNode head = new ListNode(1);
21     head.next = new ListNode(2);
22     head.next.next = new ListNode(3);
23     head.next.next.next = new ListNode(4);
24     head.next.next.next.next = new ListNode(5);
25     head.next.next.next.next.next = new ListNode(6);
26     head.next.next.next.next.next.next = new ListNode(7);
27     head.next.next.next.next.next.next.next = new ListNode(8);
28
29     ListNode result = ReverseEveryKElements.reverse(head, 3);
30     System.out.print("Nodes of the reversed LinkedList are: ");
31     while (result != null) {
32         System.out.print(result.value + " ");
33         result = result.next;
34     }
35 }
36 }

```

Run

Save

Reset



Solution

The problem follows the **In-place Reversal of a LinkedList** pattern and is quite similar to [Reverse a Sub-list](#). The only difference is that we have to reverse all the sub-lists. We can use the same approach, starting with the first sub-list (i.e. $p=1$, $q=k$) and keep reversing all the sublists of size 'k'.

Code

Most of the code is the same as [Reverse a Sub-list](#); only the highlighted lines have a majority of the changes:

Java

Python3

C++

JS

```

1  import java.util.*;
2
3  class ListNode {
4      int value = 0;
5      ListNode next;
6
7      ListNode(int value) {
8          this.value = value;
9      }
10 }
11
12 class ReverseEveryKElements {
13
14     public static ListNode reverse(ListNode head, int k) {
15         if (k <= 1 || head == null)
16             return head;
17
18         ListNode current = head, previous = null;
19         while (true) {
20             ListNode lastNodeOfPreviousPart = previous;
21             // after reversing the LinkedList 'current' will become the last node of the sub-list
22             ListNode lastNodeOfSubList = current;

```

```

23     ListNode next = null; // will be used to temporarily store the next node
24     // reverse 'k' nodes
25     for (int i = 0; current != null && i < k; i++) {
26         next = current.next;
27         current.next = previous;
28         previous = current;
29         current = next;
30     }
31
32     // connect with the previous part
33     if (lastNodeOfPreviousPart != null)
34         lastNodeOfPreviousPart.next = previous; // 'previous' is now the first node of the sub-list
35     else // this means we are changing the first node (head) of the LinkedList
36         head = previous;
37
38     // connect with the next part
39     lastNodeOfSubList.next = current;
40
41     if (current == null) // break, if we've reached the end of the LinkedList
42         break;
43     // prepare for the next sub-list
44     previous = lastNodeOfSubList;
45 }
46
47 return head;
48 }
49
50 public static void main(String[] args) {
51     ListNode head = new ListNode(1);
52     head.next = new ListNode(2);
53     head.next.next = new ListNode(3);
54     head.next.next.next = new ListNode(4);
55     head.next.next.next.next = new ListNode(5);
56     head.next.next.next.next.next = new ListNode(6);
57     head.next.next.next.next.next.next = new ListNode(7);
58     head.next.next.next.next.next.next.next = new ListNode(8);
59
60     ListNode result = ReverseEveryKElements.reverse(head, 3);
61     System.out.print("Nodes of the reversed LinkedList are: ");
62     while (result != null) {
63         System.out.print(result.value + " ");
64         result = result.next;
65     }
66 }
67 }

```

Run

Save

Reset



Time complexity

The time complexity of our algorithm will be $O(N)$ where 'N' is the total number of nodes in the LinkedList.

Space complexity

We only used constant space, therefore, the space complexity of our algorithm is $O(1)$.

[< Back](#)
☒ Mark As Completed

[Next >](#)

Reverse a Sub-list (medium)

Problem Challen



